Learning Objectives

Learners will be able to...

- **Identify the fundamental characteristics and applications of B-Trees, distinguishing them from other tree-based data structures.**
- **Analyze the structural elements and key properties of B-Trees, including node organization, branching factor, and balance criteria.**
- **Execute the process of inserting keys into a B-Tree, applying the principles of node splitting and maintaining tree balance.**
- **Implement the deletion operation in B-Trees, ensuring the tree remains balanced by correctly handling node merging and redistribution.**
- **Evaluate the time and space complexity of operations in B-Trees, including search, insertion, and deletion, to assess their efficiency in data management scenarios.**

Introduction to B-Trees

B-Trees are a type of self-balancing tree data structure that maintains sorted data in a way that allows for efficient insertion, deletion, and lookup operations. They are particularly well-suited for storage systems that read and write large blocks of data, such as databases and file systems.

Key Characteristics

- **Balanced Tree**: B-Trees are balanced, meaning the path from the root to any leaf is approximately the same length, ensuring efficient operations.
- **Multi-way Tree**: Unlike binary search trees, a B-Tree node can have more than two children.
- **Sorted Order**: Each node's keys are kept in sorted order, and each key separates the data ranges of its two children.

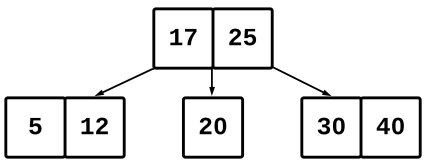
Structure of B-Trees

A B-Tree of order `m` has the following properties:

- Each node has at most `m` children.
- Each internal node (except the root) has at least `⌈m/2⌉` children.
- The root has at least two children if it is not a leaf node.
- A non-leaf node with `k` children contains `k-1` keys.

Example of a B-Tree

Consider a B-Tree of order 3 (each node can have a maximum of 3 children). Here's a simple representation:



In this example:

- The root node contains two keys (17 and 25) and has three children.
- The left child of the root contains keys 5 and 12.
- The middle child of the root contains the key 20.
- The right child of the root contains the keys 30 and 40.

Applications of B-Trees

B-Trees are widely used in databases and file systems due to their ability to efficiently handle large amounts of data. Their structure allows for minimal disk reads and writes, making them ideal for storage systems where data is not primarily kept in memory.

Summary

B-Trees offer a balanced and efficient structure for sorting, searching, and sequential access of data. Their ability to handle large blocks of data with minimal disk I/O makes them a cornerstone of modern database and file system design. In the following lessons, we will delve deeper into the specific operations of insertion, deletion, and analyze their time and space complexity.

Structure and Properties of B-Trees

B-Trees are a type of self-balancing tree data structure that maintains sorted data and allows searches, sequential access, insertions, and deletions in logarithmic time. They are particularly well-suited for storage systems that read and write large blocks of data.

Key Properties

1. **Balanced**: B-Trees are always balanced, meaning the path from the root to any leaf is the same length.
2. **Order of the Tree**: The order (or degree) of a B-Tree is the minimum number of children each non-leaf node must have, except for the root. For a B-Tree of order `m`, each node can have a maximum of `m` children.
3. **Keys in Nodes**: Each node can have a maximum of `m-1` keys and a minimum of `⌈m/2⌉ - 1` keys, except for the root node. The minimum number of keys in a node is often called the degree.
4. **Sorted Keys**: Keys within a node are always sorted in a specific order (e.g., ascending).
5. **Leaf Nodes at Same Level**: All leaf nodes are at the same level.

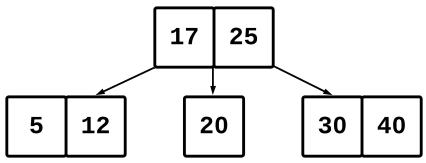
Structure of a B-Tree Node

A B-Tree node typically contains:

- An array of keys.
- An array of child pointers.
- A count of the number of keys.

Example of a B-Tree

Consider a B-Tree of order 3:



- Each node has at most 3 children (order 3).
- The root has 2 keys (`17`, `25`), and each key separates three subtrees.
- All leaf nodes are at the same level.

Summary

The structure and properties of B-Trees make them ideal for systems that handle large blocks of data, such as databases and file systems. Their ability to remain balanced with every insertion and deletion ensures efficient operations. Understanding these properties is crucial for implementing and utilizing B-Trees effectively. In the next lesson, we will delve into the insertion process in B-Trees and how it adheres to these properties to maintain the tree's balanced nature.

Implementing a B-Tree

B-Tree Nodes

B-Trees, unlike red-black and AVL trees, are not a type of BST. They are, however, a form of a binary tree, meaning children to the left have lower values and children on the right have greater values. Because of this, our node is going to look different from what we have seen in the past few assignments. A B-Tree nodes have multiple keys (values stored) and multiple children. To keep things simple, we are going to use C++'s `vector` data type for the keys and children. We also need to keep track of the leaf nodes, which will be done with a boolean value. In the constructor, set the `leaf` attribute to the boolean parameter. The keys and children will be empty vectors.

```
class BTreeNode {
public:
    std::vector<int> keys;
    std::vector<BTreeNode*> children;
    bool leaf;

    BTreeNode(bool l) : leaf(l) {}
};
```

B-Tree Class

Our B-Tree needs a root node. We also need a degree, or minimum number of keys in a node. B-Trees also have an order and a maximum number of keys. When creating a new `BTree` object, pass it the desired degree. Create the root node, which is also a leaf node since there are no other nodes in the tree. We also need to calculate the order of the tree and maximum number of keys a node can store.

```
class BTree {
private:
    BTreeNode* root;
    int degree;
    int order;
    int maxKeys;

public:
    BTree(int d) {
        degree = d;
        order = degree * 2;
        maxKeys = order - 1;
        root = new BTreeNode(true);
    }

    void print();
};
```

Because B-Trees have a different structure compared to BSTs (and its variants), we are going to create a `print` method that displays a level by level, node by node representation. To do this, we are going to use C++'s `Queue` data structure. Start by checking if the tree is empty. if so, print a message to the user. Then create a queue and add the root to the queue.

```
public:
  void print() {
      if (root == nullptr) {
          std::cout << "The tree is empty." << std::endl;
          return;
  }

  std::queue<BTreeNode*> q;
  q.push(root);
```

As long as the queue is not empty, create the `leveSize` variable which is equal to the size of the queue. Create a temporary B-Tree node and set its value to the next element in the queue. Iterate over the keys in the temporary node, printing each key followed by a space. Once you have printed all of the keys, print the `|` character followed by a space. This will help us differentiate between the different nodes on each level of the tree.

```
public:
  void print() {
      if (root == nullptr) {
          std::cout << "The tree is empty." << std::endl;
          return;
      }

      std::queue<BTreeNode*> q;
      q.push(root);

      while (!q.empty()) {
          int levelSize = q.size();

          for (int i = 0; i < levelSize; ++i) {
              BTreeNode* currentNode = q.front();
              q.pop();

              // Print keys in the current node
              for (int key : currentNode->keys) {
                  std::cout << key << " ";
              }
              std::cout << "| ";

              // Enqueue child nodes
              for (BTreeNode* child : currentNode->children) {
                  if (child != nullptr) {
                      q.push(child);
                  }
              }
          }

          std::cout << std::endl; // Move to the next level
      }
  }
```

Before moving on to the next node in the tree, you want to load all of the current node's children into the queue so they can be printed later. Finally, use the `std::cout << std::endl` command to move to the next line for our output. This ensures that each level of the tree is on its own line of output.

```
public:
    void print() {
        if (root == nullptr) {
            std::cout << "The tree is empty." << std::endl;
            return;
        }

        std::queue<BTreeNode*> q;
        q.push(root);

        while (!q.empty()) {
            int levelSize = q.size();

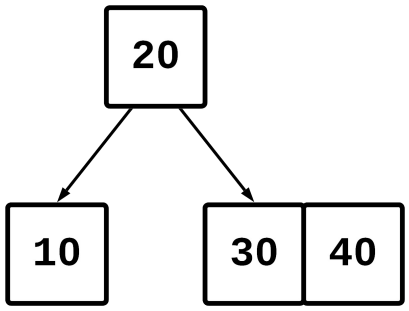
            for (int i = 0; i < levelSize; ++i) {
                BTreeNode* currentNode = q.front();
                q.pop();

                // Print keys in the current node
                for (int key : currentNode->keys) {
                    std::cout << key << " ";
                }
                std::cout << "| ";

                // Enqueue child nodes
                for (BTreeNode* child : currentNode->children) {
                    if (child != nullptr) {
                        q.push(child);
                    }
                }
            }

            std::cout << std::endl; // Move to the next level
        }
    }
};
```

We have not yet implemented an insertion, so we cannot test the `print` method right now. However, we can discuss what future output will look like. Imagine that we have the following B-Tree:



This tree has two level. The first level has the root node, which has a single key `20`. The second level has two nodes. The first node has single key `10`. The second node has the keys `30` and `40`. The `print` method above would return:

```
20 |
10 | 30 40 |
```

Each line of output represents a level in the tree. The `|` characters separate the nodes. So the first level is the root, which has the key `20`. The first child has only one key, `10`. The second child has two keys, `30`, and `40`. We will be using this format to express our B-Trees after insertion and deletion.

Add the main method to the BTreeImplementation class. In it, instantiate a BTree object and give it a degree of `2`. That means the order is `4` and the maximum number of keys per node is `3`. Without the ability to insert a node into the tree, we cannot print anything. However, our code should still compile without error.

```
int main() {
    BTree tree(2);

    return 0;
}
```

▼ Code

Your code should look like this:

```
class BTreeNode {
public:
    std::vector<int> keys;
    std::vector<BTreeNode*> children;
    bool leaf;

    BTreeNode(bool l) : leaf(l) {}
};

class BTree {
private:
    BTreeNode* root;
    int degree;
    int order;
    int maxKeys;

public:
    BTree(int d) {
        degree = d;
        order = degree * 2;
        maxKeys = order - 1;
        root = new BTreeNode(true);
    }
    void print() {
        if (root == nullptr) {
```

```
            std::cout << "The tree is empty." << std::endl;
            return;
        }

        std::queue<BTreeNode*> q;
        q.push(root);

        while (!q.empty()) {
            int levelSize = q.size();

            for (int i = 0; i < levelSize; ++i) {
                BTreeNode* currentNode = q.front();
                q.pop();

                // Print keys in the current node
                for (int key : currentNode->keys) {
                    std::cout << key << " ";
                }
                std::cout << "| ";

                // Enqueue child nodes
                for (BTreeNode* child : currentNode->children) {
                    if (child != nullptr) {
                        q.push(child);
                    }
                }
            }

            std::cout << std::endl; // Move to the next level
        }
    }
};


int main() {
    BTree tree(2);

    return 0;
}
```

You should see the following output:

```
Command was successfully executed.
```

Insertion in B-Trees

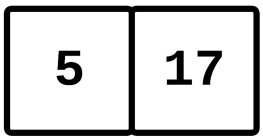
Insertion in B-Trees is a critical operation that maintains the tree's balanced structure while accommodating new elements. This process involves several steps to ensure that the tree adheres to its defining properties.

Steps for Insertion

1. **Start at the Root**:
    - Begin at the root and traverse down the tree to find the appropriate leaf node where the new key should be inserted.
2. **Locating the Correct Node**:
    - Compare the key to be inserted with the keys in the current node, moving to the appropriate child node each time.
    - The goal is to find a leaf node where the key fits according to the tree's order.
3. **Simple Insertion**:
    - If the identified leaf node has fewer than `m-1` keys (where `m` is the order of the tree), insert the new key in the correct position to maintain the sorted order.
4. **Handling Overflow - Splitting the Node**:
    - If the leaf node is full, it needs to be split after inserting the new key.
    - Create two new nodes and distribute the keys evenly. The middle key is promoted to the parent node.
    - If the parent node is also full, repeat the splitting process up the tree.
5. **Creating a New Root**:
    - If the root node splits, a new root is created with the middle key from the split.

Example of Insertion

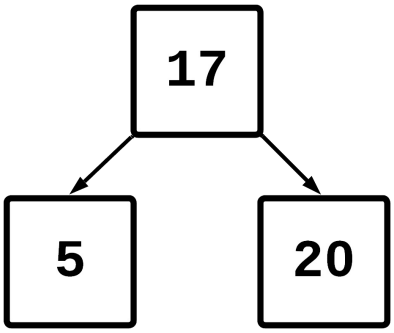
Consider a B-Tree of order 3, which means that any given node can store no more than two keys. Assume that the root has the keys `5` and `17`.



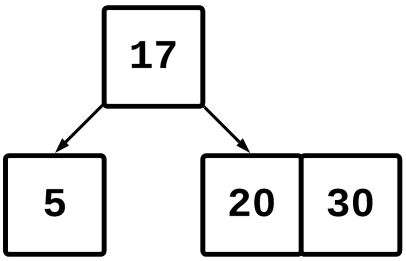
We then add the key `20` to the node. Arrange the keys in order. This node is in violation because it has too many keys. Find the middle key.



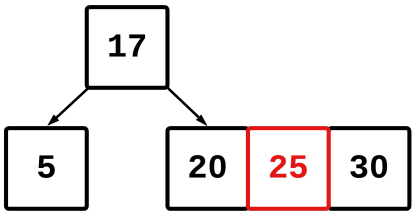
Take the middle key and put it into the node one level above. Since there are no nodes above the root, we have to make a new node and put `17` in it. This new node is now the root. Split keys `5` and `20` into their own node.



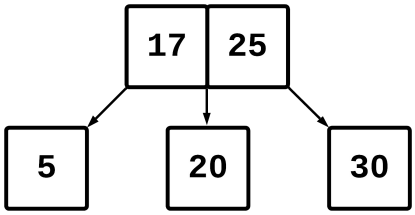
Now imagine we want to add key `30` to the tree. Since `30` is greater than `17`, it goes into the second child. Arrange the keys in order.



Now add the key `25` to the tree. It would go into the second child. This, however, leads to a violation. Identify the middle key.



Move `25` up to the root. Split the child so that keys `20` and `30` are in their own nodes.



Summary

Insertion in B-Trees is a delicate process that ensures the tree remains balanced and adheres to its properties. By understanding these steps, we can appreciate the efficiency of B-Trees in managing large datasets. The next lesson will delve into the deletion process in B-Trees, which is equally crucial for maintaining the tree's structure.

Implementing Insertion

Inserting a Node

We are going to create three methods to handle inserting a node into a B-Tree. The first method is called `insert`. It takes a key to be inserted into the tree. Create the variable `r` and assign it the value of the root node. If `r` is full (the number of keys in the `keys` list is equal to `maxKeys`), then we need to split `r`. Start by creating a new root. Assign the `root` attribute to the new root. Add the old root (`r`) as a child to the new root. Call the `splitChild` method and pass it the new root and index `0`. After we split the child, we can call `insertNonFull` and pass it the new root and the key to be inserted. If `r` is not full, call the `insertNonFull` method and pass it `r` and the value to be inserted.

```
public:
    void insert(int key) {
        BTreeNode* r = root;
        if (r->keys.size() == maxKeys) {
            BTreeNode* newRoot = new BTreeNode(false);
            root = newRoot;
            newRoot->children.push_back(r);
            splitChild(newRoot, 0);
            insertNonFull(newRoot, key);
        } else {
            insertNonFull(r, key);
        }
    }
```

When we are inserting a key into a node that is not full, we must first determine the index (`i`) of the last element in the keys list. In a `B-Tree`, keys can only be inserted into a `leaf` node. Create a conditional that checks if the current node is a `leaf` node. If yes, iterate backwards through the keys list. At each element determine if the `index` is still in the list and if the key to insert is less than the element at index `i`. If so, decrement `i` by `1`. Then add the key to the keys list at index `i + 1`.

```
private:
    void insertNonFull(BTreeNode* node, int key) {
        int i = node->keys.size() - 1;
        if (node->leaf) {
            while (i >= 0 && key < node->keys[i]) {
                i--;
            }
            node->keys.insert(node->keys.begin() + i + 1, key);
        }
    }
```

If the current node is not a `leaf` node, use a while loop to iterate backward over the keys list. Do this as long as the index is still in the list and the key to insert is less than the element at index `i`. Once this loop stops, increment `i` by `1`. The index `i` now represents the child in which the key will be inserted. However, we must first check to see if this child node is full. If so, split the node. If the key to be inserted is greater than the key at index `i`, increment `i` by `1`. Finally, recursively call `insertNonFull` with the child at index `i` and the key to be inserted.

```
private:
    void insertNonFull(BTreeNode* node, int key) {
        int i = node->keys.size() - 1;
        if (node->leaf) {
            while (i >= 0 && key < node->keys[i]) {
                i--;
            }
            node->keys.insert(node->keys.begin() + i + 1, key);
        } else {
            while (i >= 0 && key < node->keys[i]) {
                i--;
            }
            i++;
            if (node->children[i]->keys.size() == maxKeys) {
                splitChild(node, i);
                if (key > node->keys[i]) {
                    i++;
                }
            }
            insertNonFull(node->children[i], key);
        }
    }
```

The third method is `splitChild`. This method takes a `B-Tree` node and an index. This `B-Tree` node is the parent of the child that is to be split. Create the temporary node `fullChild`, which is the node that needs to be split since it is full. We also need a new child node that is empty. Add the new, empty child node to the `children` list in the parent node. Find the median node in the child that is full and add it to the `keys` list in the parent node.

```
private:
    void splitChild(BTreeNode* parentNode, int i) {
        // full child that can't take another value
        BTreeNode* fullChild = parentNode->children[i];

        // new, empty child
        BTreeNode* newChild = new BTreeNode(fullChild->leaf);

        // add new, empty child as a child to the parent
        parentNode->children.insert(parentNode->children.begin()
        + i + 1, newChild);

        // add median value from the full child to the parent
        parentNode->keys.insert(parentNode->keys.begin() + i,
        fullChild->keys[degree - 1]); // median key
    }
```

Now that the median key is in the parent node, we can split the `fullchild`. The median value serves to divide the keys list into the `first` half and second half. We are going to copy the `second` half of the list to the new node and then erase the second half of the list. Now we have two child nodes that are half-full.

To do this, use the copy and insert methods on the `fullchild` and select the keys from degree to `maxKeys`. This will select the keys from the `second` half of the list. Then use the clear method to erase these keys from `fullChild` as they are now in the new child. Finally, if the child that was split was not a `leaf` node, then we need to update the `children`.

```
private:
    void splitChild(BTreeNode* parentNode, int i) {
        // full child that can't take another value
        BTreeNode* fullChild = parentNode->children[i];

        // new, empty child
        BTreeNode* newChild = new BTreeNode(fullChild->leaf);

        // add new child as a child to the parent
        parentNode->children.insert(parentNode->children.begin()
        + i + 1, newChild);

        // add median value from the full child to the parent
        parentNode->keys.insert(parentNode->keys.begin() + i,
        fullChild->keys[degree - 1]); // median key

        // new child gets the second half of the full child
        newChild->keys.insert(newChild->keys.end(), fullChild-
        >keys.begin() + degree, fullChild->keys.end());

        // clear the second half of the full child
        fullChild->keys.erase(fullChild->keys.begin() + degree -
        1, fullChild->keys.end());

        // if the child that was split is not a leaf update the
        children
        if (!fullChild->leaf) {
            newChild->children.insert(newChild->children.end(),
        fullChild->children.begin() + degree, fullChild-
        >children.end());
            fullChild->children.erase(fullChild-
        >children.begin() + degree, fullChild->children.end());
        }
    }
```

Update the `main` method so that we add the nodes `10`, `20`, `30`, and `40` to the tree. Since the order is `4`, that means the tree can only hold three nodes. By adding four nodes, we can test that the `splitChild` method is working as expected. Then print the list.

```
int main() {
    BTree tree(2);

    tree.insert(10);
    tree.insert(20);
    tree.insert(30);
    tree.insert(40);

    tree.print();

    return 0;
}
```

You should see the following output:

```
20 |
10 | 30 40 |
```

▼ **Code**

Your code should look like this:

```
class BTreeNode {
public:
    std::vector<int> keys;
    std::vector<BTreeNode*> children;
    bool leaf;

    BTreeNode(bool l) : leaf(l) {}
};

class BTree {
private:
    BTreeNode* root;
    int degree;
    int order;
    int maxKeys;

public:
    BTree(int d) {
        degree = d;
        order = degree * 2;
        maxKeys = order - 1;
        root = new BTreeNode(true);
    }
    void print() {
        if (root == nullptr) {
            std::cout << "The tree is empty." << std::endl;
            return;
        }

        std::queue<BTreeNode*> q;
        q.push(root);

        while (!q.empty()) {
            int levelSize = q.size();

            for (int i = 0; i < levelSize; ++i) {
                BTreeNode* currentNode = q.front();
                q.pop();

                // Print keys in the current node
                for (int key : currentNode->keys) {
                    std::cout << key << " ";
                }
                std::cout << "| ";

                // Enqueue child nodes
                for (BTreeNode* child : currentNode-
        >children) {
```

```
                    if (child != nullptr) {
                        q.push(child);
                    }
                }
            }

            std::cout << std::endl; // Move to the next
        level
        }
    }
      void insert(int key) {
          BTreeNode* r = root;
          if (r->keys.size() == maxKeys) {
              BTreeNode* newRoot = new BTreeNode(false);
              root = newRoot;
              newRoot->children.push_back(r);
              splitChild(newRoot, 0);
              insertNonFull(newRoot, key);
          } else {
              insertNonFull(r, key);
          }
      }
private:
    void insertNonFull(BTreeNode* node, int key) {
        int i = node->keys.size() - 1;
        if (node->leaf) {
            while (i >= 0 && key < node->keys[i]) {
                i--;
            }
            node->keys.insert(node->keys.begin() + i + 1,
        key);
        } else {
            while (i >= 0 && key < node->keys[i]) {
                i--;
            }
            i++;
            if (node->children[i]->keys.size() == maxKeys)
        {
                splitChild(node, i);
                if (key > node->keys[i]) {
                    i++;
                }
            }
            insertNonFull(node->children[i], key);
        }
    }

    void splitChild(BTreeNode* parentNode, int i) {
        // full child that can't take another value
        BTreeNode* fullChild = parentNode->children[i];

        // new, empty child
        BTreeNode* newChild = new BTreeNode(fullChild-
        >leaf);

        // add new child as a child to the parent
        parentNode->children.insert(parentNode-
        >children.begin() + i + 1, newChild);
```

```
        // add median value from the full child to the
        parent
        parentNode->keys.insert(parentNode->keys.begin() +
        i, fullChild->keys[degree - 1]); // median key

        // new child gets the second half of the full child
        newChild->keys.insert(newChild->keys.end(),
        fullChild->keys.begin() + degree, fullChild-
        >keys.end());

        // clear the second half of the full child
        fullChild->keys.erase(fullChild->keys.begin() +
        degree - 1, fullChild->keys.end());

        // if the child that was split is not a leaf update
        the children
        if (!fullChild->leaf) {
            newChild->children.insert(newChild-
        >children.end(), fullChild->children.begin() +
        degree, fullChild->children.end());
            fullChild->children.erase(fullChild-
        >children.begin() + degree, fullChild-
        >children.end());
        }
    }
};

int main() {
    BTree tree(2);

    tree.insert(10);
    tree.insert(20);
    tree.insert(30);
    tree.insert(40);

    tree.print();

    return 0;
}
```

Implementing Search

Searching a B-Tree

We are first going to create a public `search` method that takes a search target. It then calls the private `search` method that takes the root node and the search target.

```
public:
    bool search(int key) {
        return search(root, key);
    }
```

Keys inserted into the `B-Tree` are done so in numerical order. Because of this, we can implement a binary search when looking for a specific key. The private search method takes a `BTreeNode` object and the search target. Calculate the `left` and `right` boundaries. As long as `left` is less than `right`, calculate the midpoint. If the key at the midpoint is equal to the search target, return true. Otherwise reconfigure the `left` and right boundaries and check the midpoint again. When the loop terminates, check to see if `left` is less than the number of children. If so, recursively call search and pass it the child node at index `left`. Otherwise, the key is not in the tree. In this case, return false.

```
private:
    bool search(BTreeNode* node, int key) {
        int left = 0;
        int right = node->keys.size() - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (key == node->keys[mid]) {
                return true;
            }

            if (key < node->keys[mid]) {
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        }

        // Search appropriate child node
        if (left < node->children.size()) {
            return search(node->children[left], key);
        }

        return false;
    }
```

Update the main method such that the `B-Tree` has the keys `10`, `20`, `30`, and `40`. Print the search results when searching for the keys `10`, `20`, `30`, `40`, and `50`.

```
int main() {
    BTree tree(2);

    tree.insert(10);
    tree.insert(20);
    tree.insert(30);
    tree.insert(40);

    std::cout << std::boolalpha;
    std::cout << tree.search(10) << std::endl;
    std::cout << tree.search(20) << std::endl;
    std::cout << tree.search(30) << std::endl;
    std::cout << tree.search(40) << std::endl;
    std::cout << tree.search(50) << std::endl; // not in tree

    return 0;
}
```

▼ Code

Your code should look like this:

```
class BTreeNode {
```

```
public:
    std::vector<int> keys;
    std::vector<BTreeNode*> children;
    bool leaf;

    BTreeNode(bool l) : leaf(l) {}
};

class BTree {
private:
    BTreeNode* root;
    int degree;
    int order;
    int maxKeys;

public:
    BTree(int d) {
        degree = d;
        order = degree * 2;
        maxKeys = order - 1;
        root = new BTreeNode(true);
    }
    void print() {
        if (root == nullptr) {
            std::cout << "The tree is empty." << std::endl;
            return;
        }

        std::queue<BTreeNode*> q;
        q.push(root);

        while (!q.empty()) {
            int levelSize = q.size();

            for (int i = 0; i < levelSize; ++i) {
                BTreeNode* currentNode = q.front();
                q.pop();

                // Print keys in the current node
                for (int key : currentNode->keys) {
                    std::cout << key << " ";
                }
                std::cout << "| ";

                // Enqueue child nodes
                for (BTreeNode* child : currentNode->children) {
                    if (child != nullptr) {
                        q.push(child);
                    }
                }
            }

            std::cout << std::endl; // Move to the next level
        }
    }
      void insert(int key) {
          BTreeNode* r = root;
```

```
            if (r->keys.size() == maxKeys) {
                BTreeNode* newRoot = new BTreeNode(false);
                root = newRoot;
                newRoot->children.push_back(r);
                splitChild(newRoot, 0);
                insertNonFull(newRoot, key);
            } else {
                insertNonFull(r, key);
            }
        }

    bool search(int key) {
        return search(root, key);
    }

private:
    void insertNonFull(BTreeNode* node, int key) {
        int i = node->keys.size() - 1;
        if (node->leaf) {
            while (i >= 0 && key < node->keys[i]) {
                i--;
            }
            node->keys.insert(node->keys.begin() + i + 1, key);
        } else {
            while (i >= 0 && key < node->keys[i]) {
                i--;
            }
            i++;
            if (node->children[i]->keys.size() == maxKeys) {
                splitChild(node, i);
                if (key > node->keys[i]) {
                    i++;
                }
            }
            insertNonFull(node->children[i], key);
        }
    }

    void splitChild(BTreeNode* parentNode, int i) {
        // full child that can't take another value
        BTreeNode* fullChild = parentNode->children[i];

        // new, empty child
        BTreeNode* newChild = new BTreeNode(fullChild->leaf);

        // add new child as a child to the parent
        parentNode->children.insert(parentNode->children.begin()
        + i + 1, newChild);

        // add median value from the full child to the parent
        parentNode->keys.insert(parentNode->keys.begin() + i,
        fullChild->keys[degree - 1]); // median key

        // new child gets the second half of the full child
        newChild->keys.insert(newChild->keys.end(), fullChild-
        >keys.begin() + degree, fullChild->keys.end());

        // clear the second half of the full child
```

```
        fullChild->keys.erase(fullChild->keys.begin() + degree -
        1, fullChild->keys.end());

        // if the child that was split is not a leaf update the
        children
        if (!fullChild->leaf) {
            newChild->children.insert(newChild->children.end(),
        fullChild->children.begin() + degree, fullChild-
        >children.end());
            fullChild->children.erase(fullChild-
        >children.begin() + degree, fullChild->children.end());
        }
    }

    bool search(BTreeNode* node, int key) {
        int left = 0;
        int right = node->keys.size() - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            if (key == node->keys[mid]) {
                return true;
            }

            if (key < node->keys[mid]) {
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        }

        // Search appropriate child node
        if (left < node->children.size()) {
            return search(node->children[left], key);
        }

        return false;
    }
};

int main() {
    BTree tree(2);

    tree.insert(10);
    tree.insert(20);
    tree.insert(30);
    tree.insert(40);

    std::cout << std::boolalpha;
    std::cout << tree.search(10) << std::endl;
    std::cout << tree.search(20) << std::endl;
    std::cout << tree.search(30) << std::endl;
    std::cout << tree.search(40) << std::endl;
    std::cout << tree.search(50) << std::endl; // not in tree

    return 0;
}
```

You should see the following output:

```
true
true
true
true
false
```

Deletion in B-Trees

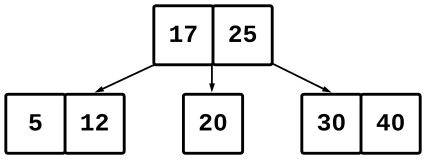
Deletion in B-Trees is a process that ensures the tree remains balanced and adheres to its properties, even after removing elements. This operation can be more complex than insertion due to the need to maintain the tree's structural integrity.

Steps for Deletion

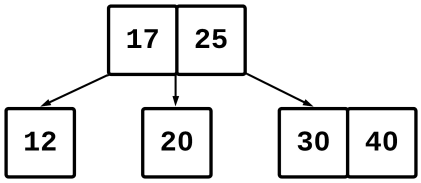
1. **Locating the Key**:
    - Start at the root and traverse down the tree to find the node containing the key to be deleted.
2. **Deleting from a Leaf Node**:
    - If the key is in a leaf node and the node has more than the minimum number of keys, simply remove the key.
3. **Deleting from an Internal Node**:
    - If the key is in an internal node, replace it with its in-order predecessor (the largest key in its left subtree) or successor (the smallest key in its right subtree). Then, delete the predecessor or successor from its original position.
4. **Handling Underflow - Merging and Borrowing**:
    - If deletion causes a node to have fewer keys than the minimum required, handle the underflow by either merging it with a sibling or borrowing a key from a sibling.
5. **Merging Nodes**:
    - If a node and its sibling both have the minimum number of keys, merge them into a single node, pulling down a key from their parent to separate the merged keys.
6. **Borrowing a Key**:
    - If a sibling has more than the minimum number of keys, borrow a key from it, adjusting the keys in the parent node accordingly.

Example of Deletion

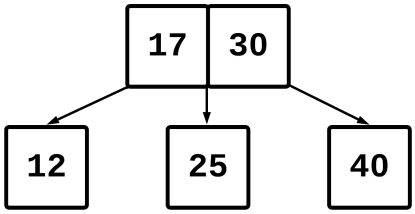
Consider a B-Tree of order 3:



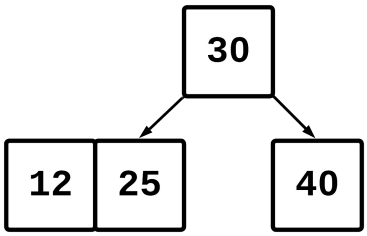
- **Deleting a Leaf Node Key**: Delete `5` from `[ 5 | 12 ]`. The node still satisfies the minimum number of keys. No other modifications need to be made to the tree.



- **Handling Underflow**: If we further delete `20`, the node `[20]` underflows. Key 25 takes the place of key 20. Key 30 takes the place of key 25.



- **Deleting an Internal Node Key**: Delete `17`. Replace it with `12` (its in-order predecessor). Then delete `17` from the leaf node. There is an underflow problem and the sibling does not have enough keys. `12` comes down from the parent and merges with `25`.



Summary

Deletion in B-Trees involves careful adjustments to ensure the tree remains balanced and adheres to its structural properties. The process can involve simple removals, replacements, merging, or borrowing, depending on the tree's state. Understanding these operations is crucial for maintaining the efficiency of B-Trees, especially in database systems and file storage applications.

Deletion Helper Methods

Helper Methods

Deleting a node from a B-Tree is more complicated than inserting a node. As such, we are going to encapsulate specific tasks of the deletion process into their own methods. Once these helper methods are done, we can tackle the deletion process as a whole. Let's start with the `deletePredecessor` method. The predecessor is the largest key in the leftmost child of a node. The first thing the method does is check to see if the node is a leaf node. If so, remove the last key. If the node is an internal node, check that the leftmost child has enough keys. If so, call `deleteSibling`, otherwise call `deleteMerge`. Finally, make a recursive call to `deletePredecessor`.

```
private:
    int deletePredecessor(BTreeNode* node) {
        if (node->leaf) {
            int lastKey = node->keys.back();
            node->keys.pop_back();
            return lastKey;
        }
        int n = node->keys.size() - 1;
        if (node->children[n]->keys.size() >= degree) {
            deleteSibling(node, n + 1, n);
        } else {
            deleteMerge(node, n, n + 1);
        }
        return deletePredecessor(node->children[n]);
    }
```

The `deleteSuccessor` method is similar to the `deletePredecessor` method. However, the successor is defined as the smallest key in the rightmost child. Start by checking if the current node is a leaf node. If so, remove the first key. If the current node is an internal node, then make sure the rightmost child has the minimum number of keys. If so, call `deleteSibling`, otherwise call `deleteMerge`. Finally, make a recursive call to `deleteSuccessor`.

```
private:
    int deleteSuccessor(BTreeNode* node) {
        if (node->leaf) {
            int firstKey = node->keys.front();
            node->keys.erase(node->keys.begin());
            return firstKey;
        }
        if (node->children[1]->keys.size() >= degree) {
            deleteSibling(node, 0, 1);
        } else {
            deleteMerge(node, 0, 1);
        }
        return deleteSuccessor(node->children[0]);
    }
```

The `deleteInternalNode` method handles the deletion of a key when it appears in an internal node. This is a recursive method, and the base case is when the current node is a `leaf` node. In this case, get the key at index `i`. If this value matches `k`, remove the key at index `i`. Get the child at index `i` and determine if it has the minimum number of keys. If so, set the key at index `i` to the result of the `deletePredecessor` method. If the child at index `i + 1` has the minimum number of keys, set the key at index `i` to the result of `deleteSuccessor` method. In all other cases, call the `deleteMerge` method and recursively call `deleteInternalNode`.

```
private:
    void deleteInternalNode(BTreeNode* node, int k, int i) {
        if (node->leaf) {
            if (node->keys[i] == k) {
                node->keys.erase(node->keys.begin() + i);
            }
            return;
        }
        if (node->children[i]->keys.size() >= degree) {
            node->keys[i] = deletePredecessor(node-
        >children[i]);
            return;
        } else if (node->children[i + 1]->keys.size() >= degree)
        {
            node->keys[i] = deleteSuccessor(node->children[i +
        1]);
            return;
        } else {
            deleteMerge(node, i, i + 1);
            deleteInternalNode(node->children[i], k, degree -
        1);
        }
    }
```

In some cases, you will need to borrow a key from a sibling node. This is handled by the `deleteSibling` method. Create a temporary variable to hold the child node at index `i`. If `i` is less than `j`, that means the sibling at index `j` has more keys. Create a temporary variable for the `right` sibling. Add the

key at index `i` of the parent node to the list of keys for the `left` child. Update the key at index `i` of the parent node with the first key of the `right` sibling. If the right sibling has children, it adds the leftmost child of the `right` sibling to the rightmost children of the `left` child. Remove the `first` key from the right sibling. If `i` is not less than `j`, that means the `left` sibling has more keys. Create a temporary variable for the `left` sibling. Now add the key at index `i-1` of the parent node to the beginning of the keys in the `left` child node (`cnode`). Update the key at index `i-1` of the parent node with the `last` key of the left sibling (`lsnode`). If the left sibling has children, add the rightmost child of the `left` sibling to the leftmost children of the `left` child (`cnode`) and remove the last key from the keys of the `left` sibling.

```
private:
    void deleteSibling(BTreeNode* node, int i, int j) {
        BTreeNode* cnode = node->children[i];
        if (i < j) {
            BTreeNode* rsnode = node->children[j];
            cnode->keys.push_back(node->keys[i]);
            node->keys[i] = rsnode->keys.front();
            if (!rsnode->children.empty()) {
                cnode->children.push_back(rsnode-
        >children.front());
                rsnode->children.erase(rsnode-
        >children.begin());
            }
            rsnode->keys.erase(rsnode->keys.begin());
        } else {
            BTreeNode* lsnode = node->children[j];
            cnode->keys.insert(cnode->keys.begin(), node->keys[i
        - 1]);
            node->keys[i - 1] = lsnode->keys.back();
            lsnode->keys.pop_back();
            if (!lsnode->children.empty()) {
                cnode->children.insert(cnode->children.begin(),
        lsnode->children.back());
                lsnode->children.pop_back();
            }
        }
    }
```

Create the `deleteMerge` method. This method merges two `child` nodes after a deletion leaves one of them with an insufficient amount of keys. The node parameter is the node with two children to be merged. Indices `i` and `j` represent the two children to be merged. If `j` is greater than `i`, this means merging with the right sibling. Create a temporary variable for the right sibling. Add the key at index `i` from the parent node to the keys of `cnode`. Iterate through the keys of the right sibling, adding each one to the keys of cnode. If the right sibling has children, add the children at index k to the list of children for `cnode`. After the loop ends, if the right sibling has children, remove the last child and add it to `cnode`. Store `cnode` in the `newNode` variable. Remove the key at index `i` in the `parent` node, and `remove` the child at index `j` in the parent node.

```
private:
    void deleteMerge(BTreeNode* node, int i, int j) {
        BTreeNode* cnode = node->children[i];
        BTreeNode* newNode;
        if (j > i) {
            BTreeNode* rsnode = node->children[j];
            cnode->keys.push_back(node->keys[i]);
            for (int k = 0; k < rsnode->keys.size(); ++k) {
                cnode->keys.push_back(rsnode->keys[k]);
                if (!rsnode->children.empty()) {
                    cnode->children.push_back(rsnode-
        >children[k]);
                }
            }
            if (!rsnode->children.empty()) {
                cnode->children.push_back(rsnode-
        >children.back());
                rsnode->children.pop_back();
            }
            newNode = cnode;
            node->keys.erase(node->keys.begin() + i);
            node->children.erase(node->children.begin() + j);
        } else {
            // Merge with left sibling
            BTreeNode* lsnode = node->children[j];
            lsnode->keys.push_back(node->keys[j]);
            for (int k = 0; k < cnode->keys.size(); ++k) {
                lsnode->keys.push_back(cnode->keys[k]);
                if (!cnode->children.empty()) {
                    lsnode->children.push_back(cnode-
        >children[k]);
                }
            }
            if (!cnode->children.empty()) {
                lsnode->children.push_back(cnode-
        >children.back());
                cnode->children.pop_back();
            }
            newNode = lsnode;
            node->keys.erase(node->keys.begin() + j);
            node->children.erase(node->children.begin() + i);
        }
        if (node == root && node->keys.empty()) {
            root = newNode;
        }
    }
```

The `findKey` method takes a `BTreeNode` and an integer representing a key for deletion. The method returns an integer, which represents the index of the key to delete. Start by creating a variable for the index and set it to 0. Iterate over the node checking to see if the index is valid and the key to delete is greater than the key at the current index. If the key is found, return the index.

```
private:
    int findKey(BTreeNode* node, int k) {
        int idx = 0;
        while (idx < node->keys.size() && node->keys[idx] < k) {
            ++idx;
        }
        return idx;
    }
```

The last helper method to create is `handleUnderflow`. This method takes a BTreeNode and an integer representing an index. This method handles cases where the number of keys is less than the degree. The method checks for four different situations. The first being the left child has at least `degree` number of keys. If so, call `deleteSibling` to borrow from the left child and redistribute the keys. The second situation is when the right child has at least `degree` keys. If so, call `deleteSibling` to borrow from the right child and redistribute the keys. The third situation is when the node's left sibling does not have enough keys and the node is not the leftmost child of its parent. In this case, call `deleteMerge` to merge the node with its left sibling. Be sure to decrement the index, otherwise you could encounter an index out of bounds error. The final situation is when the node is the leftmost child of its parent and the right sibling does not have enough keys. In this case, merge the node with the right sibling by calling the `deleteMerge` method.

```
private:
  void handleUnderflow(BTreeNode* node, int idx) {
        if (idx != 0 && node->children[idx - 1]->keys.size() >=
        degree) {
            deleteSibling(node, idx, idx - 1);
        } else if (idx + 1 < node->children.size() && node-
        >children[idx + 1]->keys.size() >= degree) {
            deleteSibling(node, idx, idx + 1);
        } else if (idx != 0) {
            deleteMerge(node, idx - 1, idx);
            idx--; // Update index if merge occurred with a left
        sibling
        } else {
            deleteMerge(node, idx, idx + 1);
        }
    }
```

Update the main method so that the B-Tree has the keys 21, 5, 99, 34, 13, 58, 19, and 17.

```
int main() {
    BTree tree(2);

    tree.insert(21);
    tree.insert(5);
    tree.insert(99);
    tree.insert(34);
    tree.insert(13);
    tree.insert(58);
    tree.insert(19);
    tree.insert(17);

    return 0;
}
```

▼ **Code**

Your code should look like this:

```
class BTreeNode {
public:
    std::vector<int> keys;
    std::vector<BTreeNode*> children;
    bool leaf;

    BTreeNode(bool l) : leaf(l) {}
};

class BTree {
private:
    BTreeNode* root;
    int degree;
    int order;
    int maxKeys;

public:
    BTree(int d) {
        degree = d;
        order = degree * 2;
        maxKeys = order - 1;
        root = new BTreeNode(true);
    }

    void print() {
        if (root == nullptr) {
            std::cout << "The tree is empty." << std::endl;
            return;
        }

        std::queue<BTreeNode*> q;
        q.push(root);

        while (!q.empty()) {
            int levelSize = q.size();
```

```
            for (int i = 0; i < levelSize; ++i) {
                BTreeNode* currentNode = q.front();
                q.pop();

                // Print keys in the current node
                for (int key : currentNode->keys) {
                    std::cout << key << " ";
                }
                std::cout << "| ";

                // Enqueue child nodes
                for (BTreeNode* child : currentNode->children) {
                    if (child != nullptr) {
                        q.push(child);
                    }
                }
            }

            std::cout << std::endl; // Move to the next level
        }
    }

      void insert(int key) {
        BTreeNode* r = root;
        if (r->keys.size() == maxKeys) {
            BTreeNode* newRoot = new BTreeNode(false);
            root = newRoot;
            newRoot->children.push_back(r);
            splitChild(newRoot, 0);
            insertNonFull(newRoot, key);
        } else {
            insertNonFull(r, key);
        }
      }

    bool search(int key) {
        return search(root, key);
    }

private:
void insertNonFull(BTreeNode* node, int key) {
    int i = node->keys.size() - 1;
    if (node->leaf) {
        while (i >= 0 && key < node->keys[i]) {
            i--;
        }
        node->keys.insert(node->keys.begin() + i + 1, key);
    } else {
        while (i >= 0 && key < node->keys[i]) {
            i--;
        }
        i++;
        if (node->children[i]->keys.size() == maxKeys) {
            splitChild(node, i);
            if (key > node->keys[i]) {
                i++;
            }
```

```
        }
        insertNonFull(node->children[i], key);
    }
}

void splitChild(BTreeNode* parentNode, int i) {
    // full child that can't take another value
    BTreeNode* fullChild = parentNode->children[i];

    // new, empty child
    BTreeNode* newChild = new BTreeNode(fullChild->leaf);

    // add new child as a child to the parent
    parentNode->children.insert(parentNode->children.begin() + i
        + 1, newChild);

    // add median value from the full child to the parent
    parentNode->keys.insert(parentNode->keys.begin() + i,
        fullChild->keys[degree - 1]); // median key

    // new child gets the second half of the full child
    newChild->keys.insert(newChild->keys.end(), fullChild-
        >keys.begin() + degree, fullChild->keys.end());

    // clear the second half of the full child
    fullChild->keys.erase(fullChild->keys.begin() + degree - 1,
        fullChild->keys.end());

    // if the child that was split is not a leaf update the
        children
    if (!fullChild->leaf) {
        newChild->children.insert(newChild->children.end(),
        fullChild->children.begin() + degree, fullChild-
        >children.end());
        fullChild->children.erase(fullChild->children.begin() +
        degree, fullChild->children.end());
    }
}

bool search(BTreeNode* node, int key) {
    int left = 0;
    int right = node->keys.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (key == node->keys[mid]) {
            return true;
        }

        if (key < node->keys[mid]) {
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }

    // Search appropriate child node
    if (left < node->children.size()) {
```

```
        return search(node->children[left], key);
    }

    return false;
}

int deletePredecessor(BTreeNode* node) {
    if (node->leaf) {
        int lastKey = node->keys.back();
        node->keys.pop_back();
        return lastKey;
    }
    int n = node->keys.size() - 1;
    if (node->children[n]->keys.size() >= degree) {
        deleteSibling(node, n + 1, n);
    } else {
        deleteMerge(node, n, n + 1);
    }
    return deletePredecessor(node->children[n]);
}


int deleteSuccessor(BTreeNode* node) {
    if (node->leaf) {
        int firstKey = node->keys.front();
        node->keys.erase(node->keys.begin());
        return firstKey;
    }
    if (node->children[1]->keys.size() >= degree) {
        deleteSibling(node, 0, 1);
    } else {
        deleteMerge(node, 0, 1);
    }
    return deleteSuccessor(node->children[0]);
}


void deleteInternalNode(BTreeNode* node, int k, int i) {
    if (node->leaf) {
        if (node->keys[i] == k) {
            node->keys.erase(node->keys.begin() + i);
        }
        return;
    }
    if (node->children[i]->keys.size() >= degree) {
        node->keys[i] = deletePredecessor(node->children[i]);
        return;
    } else if (node->children[i + 1]->keys.size() >= degree) {
        node->keys[i] = deleteSuccessor(node->children[i + 1]);
        return;
    } else {
        deleteMerge(node, i, i + 1);
        deleteInternalNode(node->children[i], k, degree - 1);
    }
}

void deleteSibling(BTreeNode* node, int i, int j) {
```

```
    BTreeNode* cnode = node->children[i];
    if (i < j) {
        BTreeNode* rsnode = node->children[j];
        cnode->keys.push_back(node->keys[i]);
        node->keys[i] = rsnode->keys.front();
        if (!rsnode->children.empty()) {
            cnode->children.push_back(rsnode->children.front());
            rsnode->children.erase(rsnode->children.begin());
        }
        rsnode->keys.erase(rsnode->keys.begin());
    } else {
        BTreeNode* lsnode = node->children[j];
        cnode->keys.insert(cnode->keys.begin(), node->keys[i -
        1]);
        node->keys[i - 1] = lsnode->keys.back();
        lsnode->keys.pop_back();
        if (!lsnode->children.empty()) {
            cnode->children.insert(cnode->children.begin(),
        lsnode->children.back());
            lsnode->children.pop_back();
        }
    }
}

void deleteMerge(BTreeNode* node, int i, int j) {
    BTreeNode* cnode = node->children[i];
    BTreeNode* newNode;
    if (j > i) {
        BTreeNode* rsnode = node->children[j];
        cnode->keys.push_back(node->keys[i]);
        for (int k = 0; k < rsnode->keys.size(); ++k) {
            cnode->keys.push_back(rsnode->keys[k]);
            if (!rsnode->children.empty()) {
                cnode->children.push_back(rsnode->children[k]);
            }
        }
        if (!rsnode->children.empty()) {
            cnode->children.push_back(rsnode->children.back());
            rsnode->children.pop_back();
        }
        newNode = cnode;
        node->keys.erase(node->keys.begin() + i);
        node->children.erase(node->children.begin() + j);
    } else {
        // Merge with left sibling
        BTreeNode* lsnode = node->children[j];
        lsnode->keys.push_back(node->keys[j]);
        for (int k = 0; k < cnode->keys.size(); ++k) {
            lsnode->keys.push_back(cnode->keys[k]);
            if (!cnode->children.empty()) {
                lsnode->children.push_back(cnode->children[k]);
            }
        }
        if (!cnode->children.empty()) {
            lsnode->children.push_back(cnode->children.back());
            cnode->children.pop_back();
        }
        newNode = lsnode;
```

```
            node->keys.erase(node->keys.begin() + j);
            node->children.erase(node->children.begin() + i);
        }
        if (node == root && node->keys.empty()) {
            root = newNode;
        }
    }

    int findKey(BTreeNode* node, int k) {
        int idx = 0;
        while (idx < node->keys.size() && node->keys[idx] < k) {
            ++idx;
        }
        return idx;
    }

    void handleUnderflow(BTreeNode* node, int idx) {
        if (idx != 0 && node->children[idx - 1]->keys.size() >=
            degree) {
            deleteSibling(node, idx, idx - 1);
        } else if (idx + 1 < node->children.size() && node-
            >children[idx + 1]->keys.size() >= degree) {
            deleteSibling(node, idx, idx + 1);
        } else if (idx != 0) {
            deleteMerge(node, idx - 1, idx);
            idx--; // Update index if merge occurred with a left
            sibling
        } else {
            deleteMerge(node, idx, idx + 1);
        }
    }

};


int main() {
    BTree tree(2);
    tree.insert(21);
    tree.insert(5);
    tree.insert(99);
    tree.insert(34);
    tree.insert(13);
    tree.insert(58);
    tree.insert(19);
    tree.insert(17);
    return 0;
}
```

We are still missing the functionality of the `deleteNode` method, so we cannot test our code right now. However, we should be able to compile our code without any errors.

```
Command was successfully executed.
```

Implementing Deletion

Now that all of the helper methods are complete, we can work on the core logic of deleting a key from a B-Tree. Start by creating the public delete method that takes a key to remove. Call the private delete method and pass it the root node and the key to delete.

```
public:
    void deleteKey(int key) {
        deleteNode(root, key);
    }
```

In the private delete method, create the variable `i` and assign it the value of the `findKey` method; `i` now represents the index of the key `k`. If the current node is a leaf node, `i` is a valid index, and the key at index `i` is the same as the key to be removed (`k`), remove the key. If the key is not found in that node, exit the method.

```
private:
    void deleteNode(BTreeNode* node, int k) {
        int i = findKey(node, k);

        if (node->leaf) {
            if (i < node->keys.size() && node->keys[i] == k) {
                node->keys.erase(node->keys.begin() + i);
            }
            return;
        }
    }
```

If the current node is an internal node and key `k` is found at index `i`, call the `deleteInternalNode` method and pass it `node`, `k`, and `i`. If key `k` is not in the current node, then it resides in one of the children. First check to see if the child node at index `i` has fewer than degree keys, call the `handleUnderflow` method. Then check to see if `i` is greater than or equal to the number of children. If this is true, then `i` is no longer a valid index. Set `i` to the number of children minus `1`. Finally, recursively call `deleteNode` and pass it the child node at index `i` and the key `k`.

```
private:
    void deleteNode(BTreeNode* node, int k) {
        int i = findKey(node, k);

        if (node->leaf) {
            if (i < node->keys.size() && node->keys[i] == k) {
                node->keys.erase(node->keys.begin() + i);
            }
            return;
        }

        if (i < node->keys.size() && node->keys[i] == k) {
            deleteInternalNode(node, k, i);
        } else {
            if (node->children[i]->keys.size() < degree) {
                handleUnderflow(node, i);
                if (i >= node->children.size()) { // Ensure i is
        still valid after underflow handling
                    i = node->children.size() - 1;
                }
            }
            deleteNode(node->children[i], k);
        }
    }
```

Update the main method to print the B-Tree after the insertions. Then remove the key `5` and print the tree again.

```
    int main() {
        BTree tree(2);

        tree.insert(21);
        tree.insert(5);
        tree.insert(99);
        tree.insert(34);
        tree.insert(13);
        tree.insert(58);
        tree.insert(19);
        tree.insert(17);

        std::cout << "Before Deletion" << std::endl;
        std::cout << "---------------" << std::endl;
        tree.print();

        std::cout << "\nDelete 5" << std::endl;
        std::cout << "--------" << std::endl;
        tree.deleteKey(5);
        tree.print();

        return 0;
    }
```

▼ Code

Your code should look like this:

```cpp
class BTreeNode {
public:
    std::vector<int> keys;
    std::vector<BTreeNode*> children;
    bool leaf;

    BTreeNode(bool l) : leaf(l) {}
};

class BTree {
private:
    BTreeNode* root;
    int degree;
    int order;
    int maxKeys;

public:
    BTree(int d) {
        degree = d;
        order = degree * 2;
        maxKeys = order - 1;
        root = new BTreeNode(true);
    }

    void print() {
        if (root == nullptr) {
            std::cout << "The tree is empty." << std::endl;
            return;
        }

        std::queue<BTreeNode*> q;
        q.push(root);

        while (!q.empty()) {
            int levelSize = q.size();

            for (int i = 0; i < levelSize; ++i) {
                BTreeNode* currentNode = q.front();
                q.pop();

                // Print keys in the current node
                for (int key : currentNode->keys) {
                    std::cout << key << " ";
                }
                std::cout << "| ";

                // Enqueue child nodes
                for (BTreeNode* child : currentNode->children) {
                    if (child != nullptr) {
                        q.push(child);
                    }
                }
            }

            std::cout << std::endl; // Move to the next level
        }
```

```
    }

      void insert(int key) {
        BTreeNode* r = root;
        if (r->keys.size() == maxKeys) {
            BTreeNode* newRoot = new BTreeNode(false);
            root = newRoot;
            newRoot->children.push_back(r);
            splitChild(newRoot, 0);
            insertNonFull(newRoot, key);
        } else {
            insertNonFull(r, key);
        }
      }

    bool search(int key) {
        return search(root, key);
    }

    void deleteKey(int key) {
        deleteNode(root, key);
    }

private:
void insertNonFull(BTreeNode* node, int key) {
    int i = node->keys.size() - 1;
    if (node->leaf) {
        while (i >= 0 && key < node->keys[i]) {
            i--;
        }
        node->keys.insert(node->keys.begin() + i + 1, key);
    } else {
        while (i >= 0 && key < node->keys[i]) {
            i--;
        }
        i++;
        if (node->children[i]->keys.size() == maxKeys) {
            splitChild(node, i);
            if (key > node->keys[i]) {
                i++;
            }
        }
        insertNonFull(node->children[i], key);
    }
}


void splitChild(BTreeNode* parentNode, int i) {
    // full child that can't take another value
    BTreeNode* fullChild = parentNode->children[i];

    // new, empty child
    BTreeNode* newChild = new BTreeNode(fullChild->leaf);

    // add new child as a child to the parent
    parentNode->children.insert(parentNode->children.begin() + i
        + 1, newChild);
```

```
    // add median value from the full child to the parent
    parentNode->keys.insert(parentNode->keys.begin() + i,
        fullChild->keys[degree - 1]); // median key

    // new child gets the second half of the full child
    newChild->keys.insert(newChild->keys.end(), fullChild-
        >keys.begin() + degree, fullChild->keys.end());

    // clear the second half of the full child
    fullChild->keys.erase(fullChild->keys.begin() + degree - 1,
        fullChild->keys.end());

    // if the child that was split is not a leaf update the
        children
    if (!fullChild->leaf) {
        newChild->children.insert(newChild->children.end(),
        fullChild->children.begin() + degree, fullChild-
        >children.end());
        fullChild->children.erase(fullChild->children.begin() +
        degree, fullChild->children.end());
    }
}

bool search(BTreeNode* node, int key) {
    int left = 0;
    int right = node->keys.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (key == node->keys[mid]) {
            return true;
        }

        if (key < node->keys[mid]) {
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }

    // Search appropriate child node
    if (left < node->children.size()) {
        return search(node->children[left], key);
    }

    return false;
}


int deletePredecessor(BTreeNode* node) {
    if (node->leaf) {
        int lastKey = node->keys.back();
        node->keys.pop_back();
        return lastKey;
    }
    int n = node->keys.size() - 1;
    if (node->children[n]->keys.size() >= degree) {
        deleteSibling(node, n + 1, n);
```

```
    } else {
        deleteMerge(node, n, n + 1);
    }
    return deletePredecessor(node->children[n]);
}


int deleteSuccessor(BTreeNode* node) {
    if (node->leaf) {
        int firstKey = node->keys.front();
        node->keys.erase(node->keys.begin());
        return firstKey;
    }
    if (node->children[1]->keys.size() >= degree) {
        deleteSibling(node, 0, 1);
    } else {
        deleteMerge(node, 0, 1);
    }
    return deleteSuccessor(node->children[0]);
}


void deleteInternalNode(BTreeNode* node, int k, int i) {
    if (node->leaf) {
        if (node->keys[i] == k) {
            node->keys.erase(node->keys.begin() + i);
        }
        return;
    }
    if (node->children[i]->keys.size() >= degree) {
        node->keys[i] = deletePredecessor(node->children[i]);
        return;
    } else if (node->children[i + 1]->keys.size() >= degree) {
        node->keys[i] = deleteSuccessor(node->children[i + 1]);
        return;
    } else {
        deleteMerge(node, i, i + 1);
        deleteInternalNode(node->children[i], k, degree - 1);
    }
}


void deleteSibling(BTreeNode* node, int i, int j) {
    BTreeNode* cnode = node->children[i];
    if (i < j) {
        BTreeNode* rsnode = node->children[j];
        cnode->keys.push_back(node->keys[i]);
        node->keys[i] = rsnode->keys.front();
        if (!rsnode->children.empty()) {
            cnode->children.push_back(rsnode->children.front());
            rsnode->children.erase(rsnode->children.begin());
        }
        rsnode->keys.erase(rsnode->keys.begin());
    } else {
        BTreeNode* lsnode = node->children[j];
        cnode->keys.insert(cnode->keys.begin(), node->keys[i -
        1]);
        node->keys[i - 1] = lsnode->keys.back();
```

```
        lsnode->keys.pop_back();
        if (!lsnode->children.empty()) {
            cnode->children.insert(cnode->children.begin(),
        lsnode->children.back());
            lsnode->children.pop_back();
        }
    }
}

void deleteMerge(BTreeNode* node, int i, int j) {
    BTreeNode* cnode = node->children[i];
    BTreeNode* newNode;
    if (j > i) {
        BTreeNode* rsnode = node->children[j];
        cnode->keys.push_back(node->keys[i]);
        for (int k = 0; k < rsnode->keys.size(); ++k) {
            cnode->keys.push_back(rsnode->keys[k]);
            if (!rsnode->children.empty()) {
                cnode->children.push_back(rsnode->children[k]);
            }
        }
        if (!rsnode->children.empty()) {
            cnode->children.push_back(rsnode->children.back());
            rsnode->children.pop_back();
        }
        newNode = cnode;
        node->keys.erase(node->keys.begin() + i);
        node->children.erase(node->children.begin() + j);
    } else {
        // Merge with left sibling
        BTreeNode* lsnode = node->children[j];
        lsnode->keys.push_back(node->keys[j]);
        for (int k = 0; k < cnode->keys.size(); ++k) {
            lsnode->keys.push_back(cnode->keys[k]);
            if (!cnode->children.empty()) {
                lsnode->children.push_back(cnode->children[k]);
            }
        }
        if (!cnode->children.empty()) {
            lsnode->children.push_back(cnode->children.back());
            cnode->children.pop_back();
        }
        newNode = lsnode;
        node->keys.erase(node->keys.begin() + j);
        node->children.erase(node->children.begin() + i);
    }
    if (node == root && node->keys.empty()) {
        root = newNode;
    }
}

int findKey(BTreeNode* node, int k) {
    int idx = 0;
    while (idx < node->keys.size() && node->keys[idx] < k) {
        ++idx;
    }
    return idx;
}
```

```
void handleUnderflow(BTreeNode* node, int idx) {
    if (idx != 0 && node->children[idx - 1]->keys.size() >=
        degree) {
        deleteSibling(node, idx, idx - 1);
    } else if (idx + 1 < node->children.size() && node-
        >children[idx + 1]->keys.size() >= degree) {
        deleteSibling(node, idx, idx + 1);
    } else if (idx != 0) {
        deleteMerge(node, idx - 1, idx);
        idx--; // Update index if merge occurred with a left
        sibling
    } else {
        deleteMerge(node, idx, idx + 1);
    }
}

private:
    void deleteNode(BTreeNode* node, int k) {
        int i = findKey(node, k);

        if (node->leaf) {
            if (i < node->keys.size() && node->keys[i] == k) {
                node->keys.erase(node->keys.begin() + i);
            }
            return;
        }

        if (i < node->keys.size() && node->keys[i] == k) {
            deleteInternalNode(node, k, i);
        } else {
            if (node->children[i]->keys.size() < degree) {
                handleUnderflow(node, i);
                if (i >= node->children.size()) { // Ensure i is
        still valid after underflow handling
                    i = node->children.size() - 1;
                }
            }
            deleteNode(node->children[i], k);
        }
    }
};

int main() {
    BTree tree(2);
    tree.insert(21);
    tree.insert(5);
    tree.insert(99);
    tree.insert(34);
    tree.insert(13);
    tree.insert(58);
    tree.insert(19);
    tree.insert(17);

    std::cout << "Before Deletion" << std::endl;
    std::cout << "---------------" << std::endl;
    tree.print();
```

```
    std::cout << "\nDelete 5" << std::endl;
    std::cout << "--------" << std::endl;
    tree.deleteKey(5);
    tree.print();

    return 0;
}
```

Key `5` is in a leaf node, but removing it will cause an underflow problem. Key `13` is now in the first child, while key `17` moves up to the root node.

```
Before Deletion
---------------
13 21 |
5 | 17 19 | 34 58 99 |

Delete 5
--------
17 21 |
13 | 19 | 34 59 99 |
```

Time and Space Complexity of B-Trees

Understanding the time and space complexity of B-Trees is crucial for evaluating their efficiency in various applications, such as databases and file systems.

Time Complexity

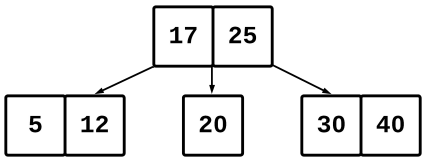
1. **Search**: The time complexity for searching a B-Tree is $O(logn)$. This is because, at each level of the tree, a binary search is performed on the keys, and the height of a B-Tree is $O(logn)$.
2. **Insertion and Deletion**: Both these operations have a time complexity of $O(logn)$. This is due to the need to traverse the tree to find the appropriate node for insertion or deletion and then perform the necessary balancing operations.

Space Complexity

The space complexity of a B-Tree is $O(n)$, where n is the number of keys stored in the tree. This is because each key occupies a certain amount of space, and the total space used is proportional to the number of keys.

Example

Consider a B-Tree of order 3:



- **Search**: To find the key `20`, we start at the root and make at most two comparisons at each level. The height of the tree is 2, so the search operation is $O(logn)$.
- **Insertion**: Inserting a new key, say `15`, involves finding the correct leaf node and then possibly splitting nodes if they overflow. The process involves traversing the tree's height, which is $O(logn)$.
- **Deletion**: Deleting a key, like `5`, might require merging or borrowing keys from sibling nodes. This operation also involves traversing the tree's height, maintaining $O(logn)$ complexity.

Summary

B-Trees are highly efficient in terms of both time and space complexity. Their logarithmic height ensures that search, insertion, and deletion operations are fast, making them ideal for systems where quick data retrieval and modification are crucial. The linear space complexity indicates that the space used grows proportionally with the number of elements, which is optimal for a data structure storing a large number of keys.

Formative assessment 1

Formative Assessment 2